

# P2206R0

## Executors Thread Pool review report

Published Proposal, 2020-08-24

### This version:

<http://wg21.link/P2206R0>

### Authors:

[Ruslan Arutyunyan](#) (Intel)

[Ben Craig](#) (NI)

[Guy Davidson](#)

[Anuya Welling](#) (Intel)

[Michael Voss](#) (Intel)

[Tom Scogland](#)

[Chris Kohlhoff](#)

[Gordon Brown](#) (Codeplay)

[Lee Howes](#) (Facebook)

### Audience:

LEWG

### Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Abstract

Authors collected the list of the issues as the result of [\[P0443R13\]](#) review with the main focus on "2.5 Thread Pools" section

## Table of Contents

### 1 Issues

#### 1.1 Design issues

#### 1.2 Wording issues

## 1.3 Issue references

### References

#### Informative References

## § 1. Issues

### § 1.1. Design issues

- ISSUE #526 ON GITHUB: “2.5.2.3 ADD ATTACH\_AND\_WAIT() METHOD”  
  
Add `attach_and_wait` method to allow the thread to be attached to the `static_thread_pool` and help with work while it's waiting for the completion
- ISSUE #528 ON GITHUB: “CONSIDER ADDING THE METHOD THAT INDICATES THAT STATIC\_THREAD\_POOL IS STOPPED”  
  
Consider adding the method that indicates that `static_thread_pool` is stopped meaning that it doesn't accept new work.
- ISSUE #529 ON GITHUB: “SCHEDULER OF STATIC\_THREAD\_POOL SHOULD HAVE THE SAME SET OF PROPERTIES AS SENDER DOES”  
  
Changes required in "2.5.2.4", "2.5.3" and "2.5.3.3"  
  
If that issue is applied need to explicitly say that `sender` returned by the `scheduler` has the same values of properties *established* as the `scheduler` itself

- ISSUE #530 ON GITHUB: “STATE THE RELATION BETWEEN THE SCHEDULER\_TYPE AND EXECUTOR\_TYPE”

Three options raised during the review:

- Always the same type
- Can be same type
- Never same type

Potential implications: constrained overload choosing. One overload is constrained by `scheduler` concept and another one by `executor` concept. If the types are the same it is always ambiguity. It can be potentially solved by introducing the different types.

Possibly there are other implications, deeper investigation is needed.

- ISSUE #535 ON GITHUB: “CONSIDER ADDING IS\_ALWAYS\_EQUAL TRAIT FOR EXECUTORS/SCHEDULERS/SENDERS”

Consider adding mentioned trait similar to the one that the allocators have to identify if schedulers/executors/senders are always equal at compile-time

- ISSUE #538 ON GITHUB: “2.2.8. REQUIRES(E &&E) SHOULD BE REQUIRES(CONST E &E)”

Please also check how it affects other parts of the paper. Probably 2.5.2.4 should be updated as well.

In 2.2.8. there is a typo as well. **S** should be used instead of **E**. So, at the end it should look like `const S&` within `requires-expression`

- ISSUE #539 ON GITHUB: “CONSIDER WHETHER TO ADD A SUBMIT MEMBER FUNCTION TO THE `STATIC_THREAD_POOL` SENDER TYPE.”

Adding submit member function to `static_thread_pool` sender type allows implementations to customize the behavior according to their needs, rather than relying on the default behavior of `std::execution::submit`.

For example implementation of `static_thread_pool` may want to customize how `sender` works with *Allocator*

- ISSUE #540 ON GITHUB: “INVESTIGATE THE NECESSITY OF `OPERATOR==` FOR `STATIC_THREAD_POOL` SENDERS AND FOR SENDERS IN GENERAL”

Probably the senders can be *EquallyComparable* optionally. If it's the case the senders could probably have special meaning as for the allocators. One of the options: "If senders are *EquallyComparable* and `operator==` returns `true` it means that they are *interchangeable*"

- ISSUE #542 ON GITHUB: “CONSIDER THIRD OPTION FOR `OUTSTANDING_WORK_T` PROPERTY”

Please investigate if it worth to have the third option for `outstanding_work_t` (like `outstanding_work.possibly` or `outstanding_work.unknown`)

- ISSUE #543 ON GITHUB: “INVESTIGATE THE RIGHT DEFAULT FOR `OUTSTANDING_WORK_T` PROPERTY”

Please take into account [#542](#)

- ISSUE #544 ON GITHUB: “CONSIDER THE THIRD OPTION FOR `RELATIONSHIP_T` PROPERTY”

Please investigate if it worth to have the third option for `relationship_t` (like `relationship.unknown`)

- ISSUE #545 ON GITHUB: “INVESTIGATE THE RIGHT DEFAULT FOR RELATIONSHIP\_T PROPERTY”

Please take into account [#544](#)

- ISSUE #552 ON GITHUB: “INVESTIGATE WHETHER VENDORS CAN ADD THEIR OWN PROPERTIES VIA FREE FUNCTIONS BUT NOT MEMBER FUNCTIONS.”

Sub-question: clarify whether implementations are allowed to provide additional overloads of [require](#).

If adding user overloads is allowed consider whether vendor properties must always result in a new executor/scheduler type.

- ISSUE #554 ON GITHUB: “INVESTIGATE THE POSSIBILITY TO SIMPLIFY ALLOCATOR\_T PROPERTY”

Please investigate if overloads with [ProtoAllocator](#) can be changed somehow to [query](#)/[require](#) both user set up allocator or default one

If the first suggestion would be applied overloads with [allocator\\_t<void>](#) seem unnecessary

- ISSUE #555 ON GITHUB: “DESCRIBE WHEN AND HOW SUBMIT/EXECUTE/OTHER SIMILAR METHODS USE ALLOCATOR”

Possible option to go is to use the same approach as *Container* requirements do.

Larger question:

- Are implementations of `static_thread_pool` allowed to do not allocate any memory at all under implementation-defined conditions (e.g. a thread pool could use a preallocated circular buffer that is "common" rather than "per function", and only touch the allocator if it runs out of space.)?
- If the `static_thread_pool` does need to allocate some per-function state that is not available, is implementation required to use the specified allocator?

Note: implementations may want to provide a submit member function for senders to avoid the heap allocation.

## § 1.2. Wording issues

- ISSUE #521 ON GITHUB: “2.5.2.3 WORKER MANAGEMENT. MAKE ATTACH LOOK LIKE THE CODE AND ADD PARENTHESES”

In `stop()` method description in the following sentence the attach mentioning should be fixed. "Subsequent invocations to **attach** complete immediately."

- ISSUE #522 ON GITHUB: “DEFINE BETTER TERM INSTEAD OF FUNCTION OBJECTS (WORK-ITEM, TASK, ETC.)”

Paper uses `function objects` in several places in the static thread pool wording. Need to define a term (like work item or task) instead of function object.

- ISSUE #523 ON GITHUB: “CONSIDER BETTER NAME FOR STATIC\_THREAD\_POOL”

`static_thread_pool` seems a bit off, since the size is sort of dynamic.

- ISSUE #524 ON GITHUB: “INVESTIGATE WHY THE WORDING HAS BEEN CHANGED AND IDENTIFY THE RIGHT ONE”

commit [9a88bfc](#)

Before:

-A `static_thread_pool`'s threads execute execution agents created via its associated executors with forward progress guarantee delegation. [*Note: Forward progress is delegated to an execution agent for its lifetime. Because `static_thread_pool` guarantees only parallel forward progress to execution agents created via its executors, forward progress delegation does not apply to execution agents which have not yet started executing their first execution step. --end note*]

After:

+A `static_thread_pool`'s threads run execution agents with forward progress guarantee delegation. [*Note: Forward progress is delegated to an execution agent for its lifetime. Because `static_thread_pool` guarantees only parallel forward progress to running execution agents; i.e., execution agents which have run the first step of the function object. --end note*]

- ISSUE #525 ON GITHUB: “PROVIDE AND EXAMPLE FOR UNTRACKED EXECUTOR WITH NO MORE WORK EXPECTED”

Provide an example where a work-item scheduled by an untracked executor can indicate that no more work is expected.

- ISSUE #527 ON GITHUB: “ADD WORDING HOW 2.5.2.3 "WORKER MANAGEMENT" FUNCTIONS INTERACT WITH EACH OTHER”

Add wording how 2.5.2.3 "Worker management" functions interact with each other and what are implications to other `static_thread_pool` methods. Take into account concurrent use-cases.

Example 1: What would happen if after `wait()` function returns user calls `executor().execute()` methods on the `static_thread_pool`` object?

Example 2: If some thread calls `wait()` and another thread calls `stop()` that would discard the work in the thread pool

- ISSUE #531 ON GITHUB: “CHANGE DESCRIPTIVE TYPE C TO SOMETHING MORE SPECIFIC”

Probably introduce the *Exposition Only* type to have better experience with reading the paper

- ISSUE #532 ON GITHUB: “REMOVE ONEWAY PROPERTY MENTIONING FROM EVERYWHERE”

- ISSUE #533 ON GITHUB: “2.5.3 SCHEDULE() METHOD SHOULD BE IN THE SYNOPSIS”

The similar modifications should be done for "Executor Types" and "Sender types" sections.

- ISSUE #534 ON GITHUB: “EXPLORE IS OPERATOR!= SHOULD BE EXPLICITLY LISTED”

Since C++20 user has `operator!=` for free if `operator==` is explicitly defined.

Request for the exploration is relevant to the whole paper where both `operator==` and `operator!=` are defined



- ISSUE #536 ON GITHUB: “REMOVE 2.5.3.5”

Related issue: [#533](#)

The similar modifications should be done for "Executor Types" and "Sender types" sections.

- ISSUE #537 ON GITHUB: “CONSIDER ALTERNATIVE NAME INSTEAD OF SCHEDULE()”

- ISSUE #541 ON GITHUB: “MAKE SURE THAT P0443 CLEARLY STATES THAT RECEIVER FUNCTIONS CANNOT BE CALLED UNTIL START IS CALLED”

- ISSUE #546 ON GITHUB: “PROVIDE STRONGER GUARANTEES ON OUTSTANDING\_WORK.UNTRACKED”

Depends on [#542](#). If the `outstanding_work.possibly` (or something) is accepted then need to make sure that `outstanding_work.unknown` provides strong enough guarantees about deadlock.

- ISSUE #547 ON GITHUB: “CONSIDER ALTERNATIVE NAMING FOR RELATIONSHIP\_T PROPERTY AND NESTED PROPERTIES FOR THAT”

- ISSUE #548 ON GITHUB: “CONSIDER ALTERNATIVE NAMING FOR OUTSTANDING\_WORK\_T PROPERTY AND NESTED PROPERTIES FOR THAT”

- ISSUE #549 ON GITHUB: “MAKE SURE P0443 DOES NOT REFER TO CONCEPT CHANGING PROPERTIES ANYMORE”

- ISSUE #550 ON GITHUB: “INTRODUCE THE "PROPERTIES" TERM OF ART IN P1393”

Consider adding *established property* term into P1393 and have the reference to that in P0443.

Related issue: [#506](#).

- ISSUE #551 ON GITHUB: “INTRODUCE THE "BEHAVIORAL PROPERTIES" TERM OF ART”

Identify if it should be added into P0443 or into P1393.

Current suggestion is to add it into P0443 because only executors introduce such model with nested properties. Probably add note that it may become the wider term and potentially may be added to P1393

- ISSUE #553 ON GITHUB: “CONSIDER CONSOLIDATION OF SCHEDULERS/SENDERS TYPE REQUIREMENTS ON THE P0443 LEVEL”

If [#529](#) is applied please find the way to consolidate schedulers/senders type requirements on the P0443 level

Something like: "The X,Y,Z classes conform to the following specification..."

Possible option to go is to use the same approach as *Container* requirements do

- ISSUE #556 ON GITHUB: “MAKE SURE THE EXCEPTION\_PTR IS THE THE ERROR TYPE FOR THE STATIC\_THREAD\_POOL SENDER”

- ISSUE #557 ON GITHUB: “CONSIDER HAVING THE DESCRIPTION OF BEHAVIORAL PROPERTIES IN ONE PLACE USE REFERENCES TO THAT”

Example: "*Returns*: An sender object of an unspecified type conforming to these specifications, associated with the same thread pool as `*this`, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned sender object. All other properties of the returned sender object are identical to those of `*this`."

It seems like the description of the *behavioral properties* mostly. It can be described in one place and then references to it can be used to do not duplicate the information

### § 1.3. Issue references

- ISSUE #506 ON GITHUB: “WHAT DOES "ESTABLISHED PROPERTY" MEAN?”

From the Executors Property review group: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2183r0.html#established>

Many places in the paper use the phrase "property already established" or something similar. But there is no precise definition of what "established" means. The paper should provide a definition, so that implementations (and users who write their own executor types) have a better understanding of what to implement.

(See the link for more discussion.)

## § References

### § Informative References

#### [P0443R13]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysisen, Carter Edwards, Gordon Brown, D. S. Hollman, Lee Howes, Kirk Shoop, Lewis Baker, Eric Niebler. [A Unified Executors Proposal for C++](#). 2 March 2020. URL: <https://wg21.link/p0443r13>